Optimising attention on a CPU

CS6886 Assignment 5 | Anjan | ME23B157

Single-head FP32 attention: scaled QK-transpose, stable row-wise softmax, then multiplication by V.

Q1. Machine and performance ceilings

Property	Measured system or specified capability
CPU	AMD Ryzen 7 7840HS, Zen 4 (Phoenix); 8 homogeneous cores, 16 SMT threads.
Clock	3.80 GHz base; up to 5.10 GHz single-core boost [1]. Host load estimate: 4.79 GHz with 1 worker; 4.56 GHz with 8.
SIMD and FMA	AVX2 and AVX-512; 32 architectural ZMM registers per thread. Two 256-bit FMA pipes; 512-bit operations use two 256-bit halves [2].
Cache	L1D 32 KiB/core; L2 1 MiB/core; shared L3 16 MiB; 64-byte lines. SMT siblings share their core caches.
Memory	2 x 16 GiB DDR5-5600 SO-DIMMs, two 64-bit channels.
Platform	HP OMEN 16-xd0017AX; Windows 11, Ubuntu 24.04 under WSL2. Clang 20.1.2, LLVM OpenMP.

Compute: 2 pipes x (256/32) lanes x 2 FLOP/FMA = 32 FLOP/cycle/core. One core at rated boost: 32 x 5.10 = **163.20 GFLOP/s**. Eight physical cores at the estimated loaded clock: 8 x 32 x 4.5564 = **1166.45 GFLOP/s**. SMT does not create additional FMA pipes.

Loaded clock estimates use host processor-performance counters (percentage of 3.801 GHz nominal) during a separate sustained FMA load. We used the median of busy-host-logical-processor samples. This is a host-derived effective clock estimate, not a direct guest clock or a guaranteed turbo rate.

Memory: 5.6 GT/s x 8 B x 2 = **89.60 GB/s** theoretical. Measured triad payload bandwidth: **37.09 GB/s** (1 thread), **39.88 GB/s** (8 threads). The triad used three 64 MiB arrays, one warm-up and nine measured sweeps, counting 12 payload bytes per element. Actual bus traffic may include write allocation.

Ridge = peak / bandwidth: using measured bandwidth gives 4.40 FLOP/B for 1 core and 29.25 for 8 cores. The theoretical-bandwidth ridges are 1.82 and 13.02 FLOP/B. Q5 uses the measured-bandwidth roofs.

[1] AMD Ryzen 7 7840HS specifications. [2] AMD Zen 4 Software Optimization Guide, document 57647, sec. 2.11 and Table 2. Installed RAM and topology were checked locally.

Q2. Naive baseline

All four supplied binary inputs were run with **make naive**. The unchanged driver times only attention (including its allocations and deallocations): one warm-up and the median of five timed calls. All variants use **-O3 -march=native** with the same compiler, and serial timings are pinned to guest CPU 0.

The driver counts **$4N^2d$ FLOP**: two dense matrix products with two FLOP per multiply-add. Softmax and packing still take time but do not add to this numerator. Percentage of peak is reported against 163.20 GFLOP/s.

N	d	Time (ms)	GFLOP/s	% one-core peak
1024	64	84.906	3.16	1.94%
2048	64	347.191	3.09	1.89%
4096	64	1481.189	2.90	1.78%
4096	128	3135.746	2.74	1.68%

Changing N and d

At d=64, doubling N increases runtime by **4.09x** and then **4.27x**. Each query considers N keys, so both products grow as N^2. At N=4096, doubling d gives **2.12x**; the number of scores is fixed but the reductions get twice as wide. Exact ratios also depend on cache reuse and run-to-run variation.

Where the time goes at N=4096, d=64

Phase	Average ms	Share of phased time	Cause
QK and scale	479.1	32.0%	Dot product for each query/key pair.
Softmax	54.4	3.6%	Max, expf, sum and normalize.
Multiply by V	965.8	64.4%	The inner j loop strides through row-major V by d floats.

Phase figures come from a separate diagnostic build with timers around each phase (mean across six calls including warm-up). They are not the reported median benchmark. The final product dominates: its strided V accesses and serial accumulation are a poor fit for SIMD and locality. The 64 MiB score matrix also exceeds the 16 MiB shared L3.

Q3. Single-threaded optimizations

a. Loop reordering (attn_loops.c)

The final multiplication changes from i,k,j to i,j,k. One score is broadcast while the feature loop walks contiguous V and O entries. Each feature is independent, so Clang vectorizes it. At the roofline input, the instrumented V phase drops from 965.8 to 154.6 ms. QK now accounts for 70% of phased time; neither its dot products nor the full score matrix changed.

b. Cache and register tiling (attn_tiled.c)

Pack K in 16-key panels so each vector contains one feature across keys. Compute scores in 32-query by 64-key cache tiles, with a four-query by 16-key register tile. Each loaded K vector feeds four queries; lanes are independent outputs. For SV, process 64-key chunks and retain output features in registers across a chunk. This version still stores the full score matrix. The diagnostic score phase (including packing) falls to 48.6 ms; scalar softmax is now the largest single phase (39%).

c. Row fusion (attn_fused.c)

Generate one score row, find its max, exponentiate, accumulate V, then scale the result by the inverse row sum. The score buffer shrinks from 64 MiB to 16 KiB at N=4096. It is reused across rows. The basic scalar QK dot products remain, and take 66% of phased time. Eliminating the large matrix reduces modeled traffic but does not remove dot products or exponential work; this explains why fusion alone is not consistently faster than loop reordering.

d. Best tested serial kernel (attn_best.c)

Combine packed four-by-16 score microtiles with a bounded query block. On AVX-512, shift logits by the row max, reduce exp arguments as $x = n \ln(2) + r$, evaluate a degree-six polynomial on $|r| <= \ln(2)/2$ and scale by 2^n. Very negative logits underflow to zero; scalar tails use expf. For SV, reuse each V vector across four queries at d=64 or two at d=128, with separate output accumulators and row normalizers.

At N=4096, d=64, the scalar-exp block-8 candidate took 124.85 ms; vector-exp blocks 4/8/16/32 took 59.82/60.34/59.84/59.72 ms (two driver medians per candidate). The selected block of 32 was marginally fastest in this screen; the small block-size differences do not establish a general ranking. Grouping the V kernel then improved the paired candidate means by 1.66x at this input. The final score and V phases take 46% and 45% of diagnostic time. Local loads, accumulation dependencies and packing remain; the peak assumes uninterrupted FMA issue.

Q3. Measurements and correctness

N / d	Variant	Time (ms)	GFLOP/s	% peak	Speedup
1024 / 64	loops	39.263	6.84	4.19%	2.16x
1024 / 64	tiled	6.002	44.73	27.41%	14.15x
1024 / 64	fused	43.060	6.23	3.82%	1.97x
1024 / 64	best	2.121	126.56	77.55%	40.03x
2048 / 64	loops	158.851	6.76	4.14%	2.19x
2048 / 64	tiled	28.225	38.04	23.31%	12.30x
2048 / 64	fused	177.070	6.06	3.71%	1.96x
2048 / 64	best	8.678	123.72	75.81%	40.01x
4096 / 64	loops	692.389	6.20	3.80%	2.14x
4096 / 64	tiled	149.318	28.76	17.62%	9.92x
4096 / 64	fused	677.651	6.34	3.88%	2.19x
4096 / 64	best	36.222	118.57	72.65%	40.89x
4096 / 128	loops	1354.531	6.34	3.88%	2.32x
4096 / 128	tiled	189.327	45.37	27.80%	16.56x
4096 / 128	fused	1372.119	6.26	3.84%	2.29x
4096 / 128	best	92.875	92.49	56.67%	33.76x

For N=4096, d=64, the final serial kernel takes **36.222 ms** versus **1481.189 ms** for naive (40.89x). Fusion is a storage change, not necessarily a speedup over the preceding variant: at N=1024, d=64, fused takes 43.060 ms versus 39.263 ms for loops.

Correctness gate

For each supplied input we checked output shape, size and finite values, then used the starter criterion: **max absolute error / max absolute reference value <= 1e-4**. We kept the timing driver unchanged and strengthened make check to require all four inputs and reject missing, invalid, or timestamp-stale outputs and builds.

All optimized serial cases and every OpenMP thread count passed. The largest error on supplied cases was **6.21e-06**. An additional 20 comparisons passed for a one-element input, odd feature width, and partial key and query blocks with large signed logits (maximum error 2.98e-07). No BLAS or external math kernels are used.

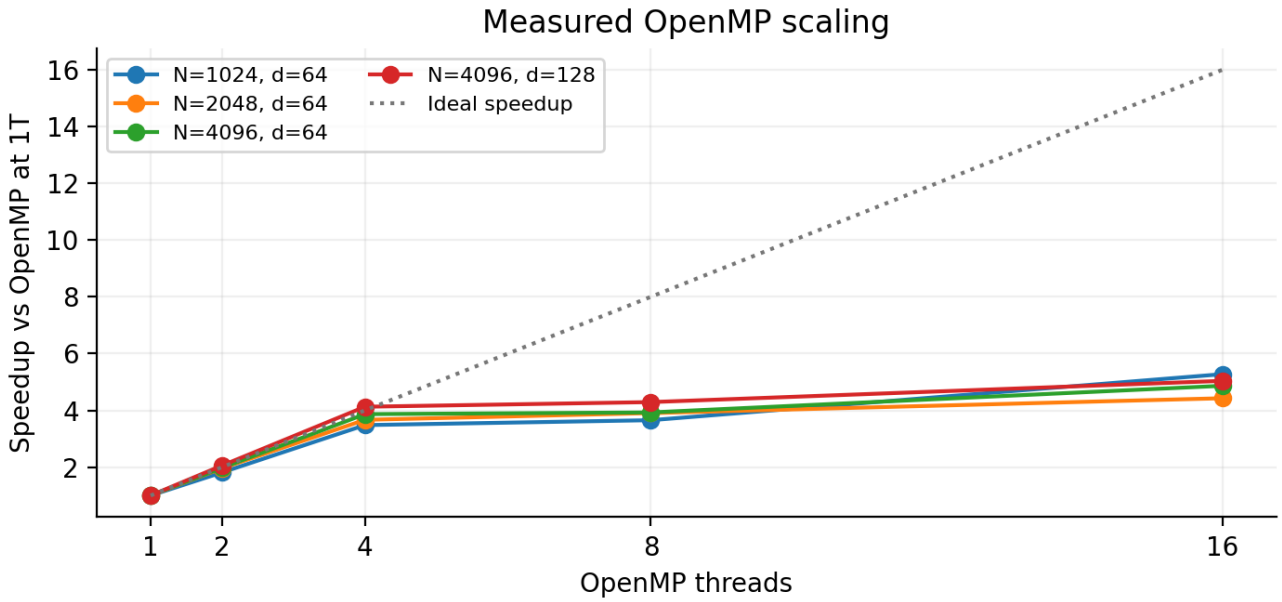
The CSV holds all measured cases. The four timing plots show each serial variant and the fastest measured OpenMP median for that input, labelled with its thread count.

Q4. OpenMP scaling

Use the best kernel unchanged inside each query block. One static OpenMP loop distributes 32-query blocks; K and V are shared read-only, each worker owns its output rows and private score scratch, and K is packed before the parallel region. The 1-8 thread runs allow one guest logical CPU per reported core; 16 threads use both SMT contexts. Host scheduling is not directly controlled.

Each entry is the median of three separate driver medians; no slow repeats were discarded. Speedup is $T_{OMP,1}/T_{OMP,p}$; efficiency is speedup/p. Each cell lists **ms / speedup / efficiency**.

Threads	1024 / 64	2048 / 64	4096 / 64	4096 / 128
1	2.068 / 1.00x / 100%	8.753 / 1.00x / 100%	35.740 / 1.00x / 100%	85.758 / 1.00x / 100%
2	1.140 / 1.81x / 91%	4.463 / 1.96x / 98%	18.085 / 1.98x / 99%	41.691 / 2.06x / 103%
4	0.594 / 3.48x / 87%	2.387 / 3.67x / 92%	9.242 / 3.87x / 97%	20.776 / 4.13x / 103%
8	0.566 / 3.65x / 46%	2.242 / 3.90x / 49%	9.100 / 3.93x / 49%	19.984 / 4.29x / 54%
16	0.392 / 5.28x / 33%	1.977 / 4.43x / 28%	7.345 / 4.87x / 30%	17.017 / 5.04x / 31%

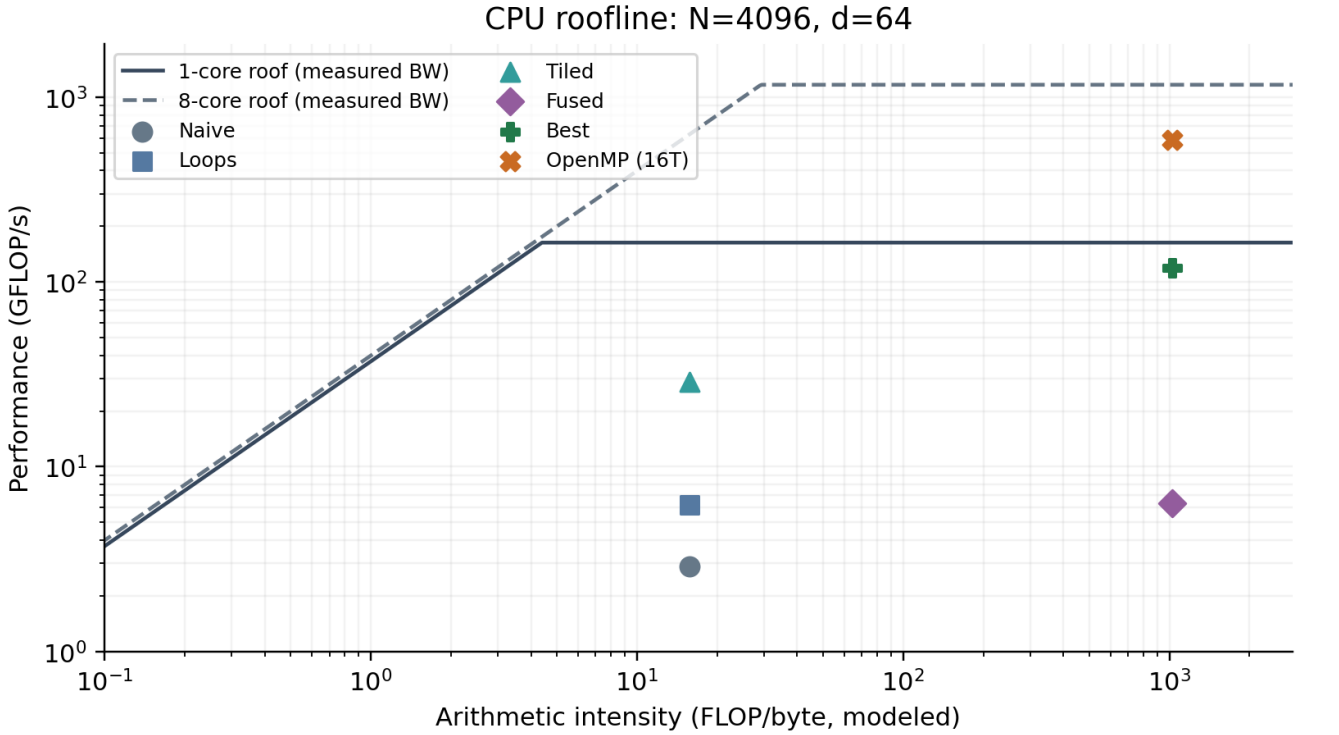


At 4096/64, the OpenMP 1-thread median is 35.740 ms versus 36.222 ms for best serial. At four threads it is 9.242 ms; at eight, 9.100 ms. The 16-thread median is 7.345 ms (4.87x versus OMP 1T), with individual driver medians from 5.722 to 10.867 ms. The overlap in observed timings makes small high-thread-count differences uncertain.

In a separate audit, the 4-, 8-, and 16-thread medians at this shape were 8.798, 8.748, and 5.333 ms. The four-to-eight plateau reproduced, but the magnitude of the 16-thread gain varied. In the same audit the streaming triad fell from 41.48 to 34.74 GB/s from 8 to 16 threads while attention got faster, so DRAM bandwidth alone does not explain the scaling. Cache contention, SMT, serial packing, and host scheduling are possible contributors; the measurements do not isolate them. The slightly above 100% entries at d=128 are relative to the observed 1-thread time, not evidence of extra compute capacity.

Q5. Roofline at N=4096, d=64

Work is $4N^2d$ = 4.295 GFLOP per call. In the traffic model Q, K, V and O contribute **4 matrices x 4Nd = 4 MiB**. For naive, loops and tiled, the 64 MiB S matrix exceeds L3: count a score write, softmax read and write, and final product read (4 x 64 MiB). Softmax row-local passes are assumed cached. For fused, best and OpenMP, row or block scores are assumed reused in cache. Packed K and private scratch are assumed cache-resident.



Variant	Modeled MiB	FLOP/byte	GFLOP/s
naive	260	15.75	2.90
loops	260	15.75	6.20
tiled	260	15.75	28.76
fused	4	1024.00	6.34
best	4	1024.00	118.57
omp (16T)	4	1024.00	584.73

Roof = min(P, BW x intensity), using Q1 values (163.2/1166.5 GFLOP/s and 37.1/39.9 GB/s). For OpenMP, use the eight-physical-core roof even when the selected run uses 16 SMT threads. The tiled variant moves upward without changing the modeled bytes; fused moves right but retains a costly QK loop. Best improves both SIMD work and reuse. OpenMP adds concurrent blocks, but does not eliminate shared-resource limits.

These are modeled inter-phase bytes, not DRAM counter measurements. The CSV applies the same accounting at each shape, even where small materialized matrices may remain cached. The model excludes write allocation, scratch eviction, and repeated Q/K/V cache traffic. The six points are right of their respective ridges under this model. At fixed work, the break-even traffic (FLOP x BW / peak) is about 931 MiB for serial and 140 MiB for OpenMP. Doubling the modeled bytes leaves the classifications unchanged; quadrupling 260 MiB puts the materialized variants on the bandwidth side. Being right of the ridge does not establish FMA saturation or identify the actual bottleneck.